

하네스 엔지니어링 및 Hook 개념 정리

본 문서는 AI 코딩 에이전트 환경에서 필수적인 '하네스 엔지니어링(Harness Engineering)'의 개념과 CI/CD, Hook, Husky 등의 핵심 요소를 정리한 대화 기록입니다.

1. 하네스 엔지니어링 개요

하네스 엔지니어링은 AI 에이전트가 의도치 않은 코드를 작성하거나 기존 시스템을 훼손하지 않도록 제어하는 **자동화된 안전망(Harness)**을 구축하는 기술입니다.

CI/CD (지속적 통합 / 지속적 배포)

- **CI (Continuous Integration):** AI 가 코드를 제출하면 자동으로 빌드 및 테스트를 수행하여 에러나 스타일 위반을 검사합니다.
- **CD (Continuous Deployment):** 검증된 코드를 실제 운영 서버에 안전하게 배포합니다.
- **역할:** AI 의 실수를 기계적으로 피드백하고 반려하는 자동 검증 파이프라인 역할을 합니다.

Hook (훅)

- 특정 이벤트 발생 직전/직후에 실행을 가로채어 검사를 수행하는 자동화 스크립트입니다.
- 보안 키 유출이나 잘못된 로그가 포함되지 않았는지 사전에 차단하는 '브레이크' 역할을 합니다.

허스크 (Husky)

- 로컬에 저장되는 Git Hook 을 프로젝트 소스 코드에 포함시켜 팀원 및 AI 에이전트와 쉽게 공유하도록 돕는 Node.js 기반 도구입니다.
- 모든 환경에서 동일한 훅 검사 규칙이 강제되도록 보장합니다.

2. Hook 과 Git Hook 의 차이점

Git Hook 은 Hook 이라는 넓은 컴퓨터 공학 개념의 하위 개념입니다.

개념	설명	예시
Hook (부모 개념)	소프트웨어 실행 도중 특정 이벤트를 가로채어 원하는 동작을 끼워 넣는 기술 전반	웹 훅(Webhook), 리액트 훅(React Hooks), 시스템 훅 등
Git Hook (자식 개념)	Git 의 이벤트(커밋, 푸시 등) 발생 전후에 자동 검증 스크립트를 실행하는 구체적인 구현체	pre-commit, pre-push 등

3. Claude 에서의 Hook

Claude Code 환경에서의 Hook 은 단순한 프롬프트 권고를 넘어, AI 에이전트의 생명주기에 개입해 **결정론적(Deterministic)으로 실행을 강제하는 자동화 장치**입니다.

주요 작동 시점

- **PreToolUse (도구 실행 직전):** AI 가 터미널 명령을 실행하거나 파일을 편집하기 직전에 발동하여, 파괴적인 명령어(예: DROP TABLE, rm -rf)를 즉시 차단(Block)합니다.
- **PostToolUse (도구 실행 직후):** 파일 작성 직후 Linter 나 테스트 코드를 실행하여, 실패할 경우 승인하지 않고 AI 에게 수정을 지시합니다.
- **SessionStart (세션 시작):** Claude 세션 시작 시 필요한 환경 변수를 주입합니다.

설정 범위 관리

- **전역 설정:** ~/.claude/settings.json
- **프로젝트 공유 설정:** .claude/settings.json (Git 공유용)
- **프로젝트 로컬 설정:** .claude/settings.local.json